

JT_DEV_B76_1.3

Local RAG knowledge search with Laravel and PostgreSQL

1 Initial situation

- 1.1 In previous projects you built applications that stored records and showed them back. Finding something in it meant knowing roughly what it was called: a `LIKE '%word%'` query, a filter, a list. In real life, this does not work if you want a useful search. Someone looking for « How do I make the container talk to the database » will never match a document titled « Service networking ».
- 1.2 Two very different techniques answer that problem. The first is full-text search: the database reduces text to normalised [lexemes](#), indexes them, and ranks documents by how well they match a query. It is exact, fast, and it is what most applications should have implemented instead of `LIKE`. The second is semantic search: a model converts text into a [vector](#) positioned so that texts about the same thing land close together. Comparing vectors then finds documents that mean the same thing without sharing a single word.
- 1.3 Put the second one in front of a language model that answers in a sentence instead of returning a list of links, and you have what's called a RAG (retrieval-augmented generation). RAG performs vector research, then uses the results to allow a model to make a precise answer.
- 1.4 In this project, you will build a simple web app with articles (provided as `.md`). You'll implement article chunking, indexing, embedding, searching... At the end you will implement a search system that uses an LLM to respond using your search system.
- 1.5 Everything in this project runs on your machine, in containers: the application, the database, and the model. This allows you to understand how all these things interact and prepare you for real-world use cases.

2 Skills from the ICT training plan worked on

Skills ranked by importance of implementation in the project: [consult the training plan](#).

- 2.1 g5 : Implémenter les applications et les interfaces selon le concept en respectant les exigences de sécurité.
- 2.2 c2 : Mettre en œuvre des modèles de données dans un dispositif de stockage de données numériques.
- 2.3 a3 : Rechercher des informations sur des solutions ICT et sur les innovations.
- 2.4 h1 : Définir la plateforme appropriée pour la livraison des applications.
- 2.5 c4 : Traiter des données à partir de dispositifs de stockage de données numériques.

3 Deliverables

- 3.1 A GitHub repository containing the Laravel application and the container configuration that runs it.
- 3.2 A `README.md` that lets your trainer clone the repository and reach a working search page, with every command listed in order, including how the models get into the running container.
- 3.3 A database schema, created by migrations, holding documents, their fragments, and the search data of both search modes.
- 3.4 A GitHub Project board with all tasks closed with GitHub close keywords.
- 3.5 A live demonstration of the search modes to your trainer, on your own machine.



4 Andragogic conditions

- 4.1 Work is carried out independently, on an individual basis.
- 4.2 You are allowed to help each other, share ideas and discuss solutions, but each trainee must provide their own implementation with their own code.
- 4.3 Apprentices have access to all the resources they need, particularly on the web.
- 4.4 AI assistance is allowed in this project, including for generating code. The condition is that you understand what you deliver: during the demonstration, your trainer will ask **why** – why this fragment size, why this threshold, why this index, why this prompt. The code you cannot explain counts as not delivered.
- 4.5 Environment and tooling: [Laravel Sail](#) for the container environment, extended with a PostgreSQL service based on the [pgvector/pgvector](#) image and an [Ollama](#) service. Embeddings and answer generation go through the [Laravel AI SDK](#) with its Ollama provider.
- 4.6 Imposed models: `nomio-embed-text` for embeddings and `llama3.2:3b` for generation. If generation latency on your machine makes the answer mode unusable, `llama3.2:1b` is the accepted fallback.
- 4.7 Access and materials: the starter repository with the text corpus; a network connection able to download roughly 2.3 GB of models on first run; a workstation with at least 16 GB of RAM.
- 4.8 The tasks of this project are described in the GitHub Project board: [Local RAG knowledge search template](#). Copy it to your own account before starting, as described in the first step below.
- 4.9 Submission is the link to your GitHub repository, the link to your GitHub Project board, and the live demonstration. Your commit history is part of the submission and will be read.
- 4.10 **Maximum allowed time for the project: 20 full days (7.5 hours per day)**. You must manage your time independently. On project beginning, inform your trainer of the start date and time.

5 Steps

- 5.1 Create a new GitHub Project from [the Local RAG knowledge search template](#) to your account (use the «...» then «Make a copy» menu). Select your account to host the project and name the project with your GitHub pseudo, for example `local-rag-<your-username>`. **Do not forget to tick the «Draft issues will be copied if selected» checkbox!**
- 5.2 Clone the [starter repository](#) provided by your trainer and read the corpus before writing any code. The shape of the data decides most of the design decisions you are about to make.
- 5.3 Bring up the environment as described in the first task of the board. Do not move on until the application, the database and the model service each answer on their own.
- 5.4 Implement the tasks of the board in order. They are sequenced. The guidance decreases as you progress, on purpose.
- 5.5 Keep notes as you go. This will help you justify choices to your trainer.
- 5.6 Book your demonstration slot with your trainer once the board is complete. **Test your full demo before !**

6 Resources

- 6.1 <https://grafikart.fr/tutoriels/rag-ia-2352>: Understand RAG by Grafikart.
- 6.2 [Laravel Sail](#): how Sail publishes its container configuration, how services are declared, and how one service reaches another. The starting point for adding a database and a model service to your environment.
- 6.3 [Define and manage volumes in Docker Compose](#): the reference for declaring persistent storage. Read it before you decide where a downloaded model lives, or you will download it again every morning.



- 6.4 [pgvector/pgvector on Docker Hub](#): the PostgreSQL images that ship the vector extension, and the tags that tell you which PostgreSQL version you get.
- 6.5 [Laravel – Database: Migrations](#): the column types and index declarations available to your schema. The vector and text-search column types are in there; find them rather than writing raw SQL by reflex.
- 6.6 [PostgreSQL – Full Text Search: Introduction](#): what a lexeme is, what the search-document and search-query types are, and why comparing raw strings is not searching.
- 6.7 [PostgreSQL – Controlling Text Search](#): the functions that build search documents, the ones that parse user queries, and the ranking functions. Several parsing functions exist, and they behave very differently on text typed by a human.
- 6.8 [PostgreSQL – Generated Columns](#): how a column can be computed and stored by the database itself. Relevant to the question of when your search data gets built.
- 6.9 [PostgreSQL – Preferred Index Types for Text Search](#): which index types serve text search, and what each one costs.
- 6.10 [PostgreSQL – Using EXPLAIN](#): how to ask the database what it actually did with your query. The only honest way to tell whether an index is being used.
- 6.11 [pgvector](#): the extension's own documentation – vector column types, the distance operators and what each one measures, the available index types, and its explicit guidance on when indexing is worth it. Read the indexing section before you add an index.
- 6.12 [Laravel – Database: Query Builder](#): the query builder methods for full-text conditions and for vector distance conditions and ordering. Read the method signatures: the default values tell you what the framework assumes.
- 6.13 [Laravel AI SDK](#): the first-party package for embeddings and text generation, its provider configuration, and its agent and tool APIs.
- 6.14 [laravel/ai on GitHub](#): the source and release notes of that package. It is a pre-1.0 release, so the repository is the reliable answer when the documentation and the version you installed disagree.
- 6.15 [Ollama – API](#): the HTTP endpoints of the model service, including how to request embeddings and how to hold a chat, and the shape of what comes back.
- 6.16 [nomic-embed-text](#): the model page of the imposed embedding model, with its size and context window.
- 6.17 [llama3.2](#): the model page of the imposed generation model and its available tags.
- 6.18 [Laravel – Queues](#): running work outside the request cycle. Worth reading before you embed a few hundred text fragments inside an HTTP request.
- 6.19 [Laravel – Eloquent: Getting Started](#): models, relations and casts, for the parts of your data access the query builder does not cover.
- 6.20 [Conventional Commits](#): the commit message convention used in this project.

7 Evaluation criteria

- 7.1 The whole environment starts without failure from a fresh clone by following the README.
- 7.2 The database schema is created by migrations, and stores the search data of both search modes.
- 7.3 Word search survives input typed by a human (empty, quoted, punctuated) without producing a database error, and ranks its results.
- 7.4 Meaning search return fragments ordered by proximity applies a threshold.
- 7.5 Answer mode answers only from retrieved fragments, names the documents it used, and refuses to answer when retrieval comes back empty.
- 7.6 The splitting strategy is documented, and its effect on retrieval is visible in the comparison.
- 7.7 Configuration lives outside the code: model names, vector size, thresholds and connection details are not hard-coded in the middle of a class.



- 7.8 The code is [clean and follows programming principles](#), follows Laravel conventions, and contains no dead code and no leftover debugging output.
- 7.9 The interface is usable without instructions: search modes are distinguishable, results readable, empty states explicit.
- 7.10 Commits follow the [conventional commits](#) standard
- 7.11 Git history shows the work progressing rather than one commit per day.
- 7.12 For each task there is a pull request, with a reviewer and a [close keyword](#).
- 7.13 All the required features are implemented as described in the project tasks.
- 7.14 During the demonstration, every design decision can be explained and defended.

